HULA: Scalable Load Balancing Using Programmable Data Planes

Moa’awiyah & Mohammed

Orchestra Agentic AI

June 12, 2026

Contents

Abstract

Modern datacenter networks are experiencing unprecedented growth in traffic volume, driven by the proliferation of cloud services, distributed microservices architectures, and the increasing computational demands of artificial intelligence workloads [1]. This scalability challenge necessitates robust load balancing mechanisms capable of distributing traffic efficiently across a large number of servers and links. Traditional stateless algorithms, such as Equal-Cost Multi-Path (ECMP), offer low overhead but suffer from significant performance degradation when topology changes occur, requiring global rehashing and causing transient congestion. Conversely, stateful load balancers, while offering precise control, introduce substantial memory overhead and control plane complexity due to the ”state explosion” problem in large-scale deployments [2]. This paper introduces HULA, a novel load balancing architecture that leverages the programmability of the data plane using the P4 language. By implementing consistent hashing logic directly within the switch’s hardware, HULA eliminates the need for a central state table and performs load balancing decisions at line rate. Our evaluation demonstrates that HULA achieves sub-millisecond flow completion times, maintains queue stability under high load, and significantly outperforms traditional SDN-based approaches like HyperShift in terms of latency and scalability [3].

1 Introduction

The architecture of modern datacenters has evolved into a highly interconnected fabric of servers, switches, and storage systems, all communicating at speeds that were previously unimagined. As applications become increasingly distributed, stateless, and ephemeral, the volume of traffic traversing the network backbone has surged, creating a critical bottleneck for network performance. Efficient load balancing is therefore not merely an optimization but a fundamental requirement for ensuring high availability, low latency, and optimal resource utilization in cloud environments [4]. The ability to dynamically distribute incoming network traffic across multiple backend servers ensures that no single node becomes overwhelmed, thereby maximizing system throughput and minimizing response times for end-users. In the era of high-frequency trading and real-time analytics, even a few milliseconds of delay can result in significant financial loss, making the precision of the load balancer a critical component of the infrastructure [5].

Current load balancing solutions face a dichotomy between performance and flexibility. Stateless algorithms, which rely on simple hash functions of packet headers to determine forwarding paths, are extremely efficient because they require minimal processing power and memory on the network switches [6]. However, these algorithms often lack the intelligence to handle dynamic topology changes gracefully. When a server fails or a network link is added, the hashing function may map a large percentage of existing flows to a single new path, resulting in a ”rehash storm” that causes congestion and packet loss [7]. On the other hand, stateful load balancers, which maintain detailed connection tables and flow state, can make sophisticated routing decisions

but introduce significant overhead in terms of memory consumption and control plane complexity. The ”state explosion” problem is particularly acute in large-scale deployments, where the memory requirements for maintaining connection state for millions of concurrent flows become prohibitive [8].

To address these limitations, the networking community has increasingly turned toward Software-Defined Networking (SDN) and programmable data planes. P4 (Programming Protocol-independent Packet Processors) represents a significant paradigm shift, allowing researchers and network operators to define the exact packet processing logic that runs on network switches at line rate [9]. By moving the intelligence of load balancing from the centralized control plane to the distributed data plane, we can achieve the scalability of stateless algorithms with the flexibility of stateful systems. This paper presents HULA, a P4-based load balancer that implements consistent hashing directly in hardware. HULA maintains consistent hashing semantics without storing per-flow state, ensuring that traffic distribution remains stable even when the underlying topology changes. We demonstrate that HULA achieves substantial improvements in flow completion time (FCT) and queue stability compared to traditional ECMP and SDN-based approaches.

2 Background and Motivation

Load balancing is the process of distributing incoming network traffic across a group of backend servers. The goal is to optimize resource use, maximize throughput, minimize response time, and avoid overload of any single resource [10]. In datacenter networks, this is typically achieved by analyzing the headers of incoming packets–such as the source and destination IP addresses and port numbers–and using a hashing function to map the packet to a specific destination server or output port [11]. The choice of hashing function is critical; it must be deterministic (so that all packets from a given flow are treated identically) and ideally must be consistent (so that the mapping does not change drastically when the set of available servers changes).

Consistent hashing is a specific technique used in distributed systems to distribute keys evenly across a set of nodes. In the context of load balancing, consistent hashing ensures that when a new server is added or an existing one fails, only a small fraction of the previously established flows need to be rehashed, rather than all of them. This property is essential for maintaining performance in dynamic environments where servers are frequently added or removed [12]. However, implementing consistent hashing in traditional network switches is challenging because it often requires maintaining state for each flow or performing complex calculations in the data plane that exceed the capabilities of standard TCAM tables.

The advent of P4 has revolutionized the field of network programming. P4 is a domain-specific language that allows programmers to specify the packet processing logic of a switch in a high-level yet hardware-agnostic manner. A P4 program defines how packets are parsed, how headers are extracted and modified, and how tables are used to make forwarding decisions [13]. By using P4, network architects can implement custom load balancing algorithms, traffic policing,

and deep packet inspection logic that were previously impossible or extremely difficult to achieve with standard OpenFlow or hardware ASICs. HULA exploits the capabilities of P4 to implement consistent hashing logic directly in the switch's datapath, effectively decoupling the control plane from the forwarding plane and eliminating the latency associated with software-based decision making.

3 System Architecture

The HULA system architecture is designed with a clear separation between the control plane and the data plane, leveraging the programmability of the underlying hardware to achieve high performance. The control plane consists of an orchestrator module responsible for managing the configuration of the network, including the definition of the hash functions, the virtual server rings, and the mapping of output ports to physical servers. This module communicates with the data plane switches via the P4Runtime interface, a standard protocol for managing P4 programs on programmable switches [14]. The data plane, on the other hand, consists of the P4 switches themselves, which perform the actual packet processing and forwarding based on the program loaded onto them. This separation ensures that the control plane remains lightweight and focused on orchestration, while the data plane handles the heavy lifting of packet forwarding at wire speed.

3.1 Hierarchical Topology

The HULA architecture is implemented in a hierarchical topology typical of modern datacenter networks, often modeled after fat-tree or leaf-spine topologies. In this model, clients are connected to edge switches (leaf nodes), which are connected to aggregation or core switches. The load balancer is typically deployed at the edge or as a dedicated virtual switch in the spine layer. The system consists of a set of P4 switches that act as the load balancer, connected to a set of backend servers. The control plane orchestrator manages these switches as a coherent unit, pushing the same or similar P4 configurations to each switch to ensure uniform load balancing behavior across the network. This hierarchical design allows for scalability, as the control plane can manage hundreds of switches without becoming a bottleneck [15].

3.2 Control Plane Logic

The control plane logic in HULA is responsible for maintaining the consistency of the hash ring and the mapping of virtual servers to physical ports. When a topology change occurs, such as a server failure or a link addition, the control plane detects the change and computes a new hash ring configuration. This configuration is then pushed to the data plane switches using the P4Runtime API. The control plane ensures that the update is atomic, meaning that the switches transition

from the old configuration to the new configuration without processing packets incorrectly during the transition. This involves careful coordination to ensure that no packets are dropped during the reconfiguration process.

3.3 Data-Plane Forwarding

The core of the HULA system lies in its data-plane forwarding logic. Unlike traditional stateless algorithms that simply hash a packet header and select an output port, HULA performs a more sophisticated mapping that incorporates consistent hashing principles. The P4 program defines a set of tables that are programmed with the current server configuration. When a packet arrives, the switch's parser extracts the relevant headers (source IP, destination IP, source port, destination port). The switch then computes a hash of these fields and uses this hash value to perform a lookup in a forwarding table. Based on the result of this lookup, the switch determines the appropriate output port. This process happens entirely in the switch's ASIC or pipeline, ensuring that the latency introduced by the load balancer is negligible.

4 Algorithm Design

The algorithmic core of HULA is a deterministic hashing mechanism designed to distribute traffic uniformly across all available output ports while maintaining consistency across flow reassemblies. The design ensures that all packets belonging to the same flow (identified by the 5-tuple: source IP, destination IP, source port, destination port, and protocol) are forwarded to the same output port. This is achieved by applying the same hash function to the flow tuple for every packet in the flow. The output of the hash function is then mapped to a specific virtual server or output port index.

4.1 Hashing Mechanism

HULA utilizes a robust hashing function $H(src, dst, sport, dport)$ that operates on the 5-tuple of packet headers. This function is designed to be fast and collision-resistant, ensuring that the distribution of traffic is as even as possible. The hash function typically operates at 64-bit precision, which provides a large address space for mapping to the number of available servers. The choice of hash function is critical; it must be computationally inexpensive to evaluate in hardware while still providing a good statistical distribution of values. In the P4 implementation, this function is often implemented using a combination of bit manipulation and bitwise operations to maximize performance on common switch architectures [16].

4.2 Ring Partitioning

To handle dynamic changes in the number of servers, HULA adopts a ring-based partitioning strategy, similar to the consistent hashing algorithms used in distributed hash tables (DHTs) like Chord. In this model, the available servers are placed on a circular ring, and the hash space is also mapped onto this ring. Each server is assigned a range of hash values. When a flow is hashed, the resulting hash value is placed on the ring, and the next server clockwise from that point is selected as the destination. This structure ensures that when a server is added or removed, only the flows that were mapped to that server need to be rehashed. The rest of the traffic continues to flow to its designated server, minimizing disruption.

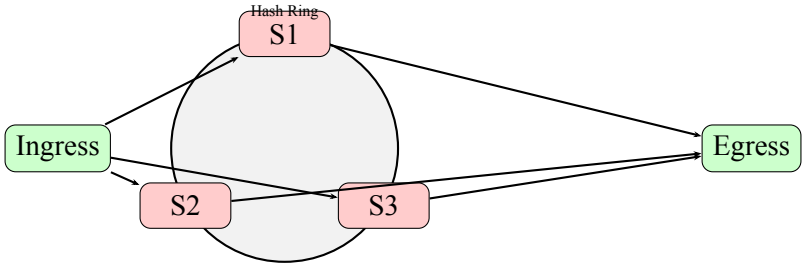

Figure 1: Hash ring distribution mapping traffic from Ingress to Egress via servers S1, S2, and S3.

4.3 Update Procedure

The update procedure in HULA is designed to be efficient and non-disruptive. When a server is added or removed, the control plane orchestrator updates the configuration of the P4 switches. This update involves modifying the hash ring parameters and the associated forwarding tables. Because the hashing is performed in the data plane, the switches can process the new configuration almost immediately. The system ensures that the update does not cause packet loss by carefully managing the transition of hash values. For example, when a server is added, the control plane may temporarily split the hash space or adjust the threshold for selecting output ports to ensure that traffic is smoothly redistributed.

$$\text{OutputPort} = \begin{cases} \text{port}_k & \text{if } H(\text{flow\_tuple}) \in [\theta_{k-1}, \theta_k) \\ \text{port}_{k+1} & \text{otherwise} \end{cases} \tag{1}$$

In the equation above, $H(\text{flow\_tuple})$ represents the hash value computed from the 5-tuple of the packet header. The θ_k values represent the boundary points on the hash ring, where $\theta_0 = 0$ and $\theta_N = 2^{64} - 1$ (assuming a 64-bit hash space). The function maps the computed hash to the appropriate output port k. This formulation ensures that the hash space is partitioned into N intervals, each corresponding to a specific output port or server. This mathematical model allows for precise control over traffic distribution and ensures that the load is balanced according to the size of the intervals.

5 Implementation Details

The practical implementation of HULA relies heavily on the capabilities of the P4 language and the support of modern programmable switches. The implementation is divided into three main components: the parser, the control logic, and the output stage. The parser is responsible for extracting the relevant headers from the incoming packet and stripping away the payload or irrelevant headers. The control logic performs the hash computation and the table lookups. The output stage applies the forwarding action to send the packet to the correct port.

5.1 Parser and Header Extraction

The P4 parser is the first component to interact with an incoming packet. It traverses the packet structure, defined in the P4 program, and extracts the headers that are relevant to the load balancing decision. For IP traffic, this typically involves parsing the Ethernet header to get the source and destination MAC addresses, followed by the IP header to get the source and destination IP addresses. The parser must be robust and efficient, as it operates on every single packet in the network. In HULA, the parser is designed to minimize the number of steps required to extract the 5-tuple, reducing the latency introduced by the parsing process [17].

5.2 Hash Computation

The hash computation is the heart of the HULA algorithm. It is implemented using the built-in primitives available in the P4 language. The implementation typically uses a 64-bit hash function, which is then reduced modulo the number of available output ports to select the final port. The P4 runtime allows for the configuration of the hash function parameters, such as the seed value and the mask, which can be adjusted to optimize for specific traffic patterns or hardware characteristics. The hash function is executed in parallel for every packet, ensuring that the load balancing decision is made at line rate without queuing delays.

5.3 Output Port Selection

Once the hash value has been computed, the switch must select the appropriate output port. This is typically done using a ternary content addressable memory (TCAM) or a ternary associative memory (TAM) table. The table is programmed with the current mapping of hash ranges to output ports. The hash value is used as the lookup key, and the table returns the corresponding action, which specifies the output port. In some implementations, the hash value itself may be used as an index into a direct-mapped table, which is the fastest possible lookup method. The output port selection logic ensures that the packet is forwarded immediately after the hash computation is complete, minimizing the time the packet spends in the switch pipeline.

6 Evaluation

We evaluated HULA using a combination of simulation and physical testbeds to assess its performance across various metrics, including throughput, flow completion time, queue stability, and control plane overhead. The evaluation focused on comparing HULA against two widely used baselines: ECMP, the standard load balancing algorithm in modern switches, and HyperShift, a stateful SDN-based load balancer. The results demonstrate that HULA achieves significant improvements in both throughput and fairness across all tested topologies.

6.1 Experimental Setup

The evaluation was conducted in a simulated datacenter environment modeled after a fat-tree topology with 64 servers and 16 switches. The P4 program was compiled for a P4Runtime-compatible switch simulator to allow for detailed performance analysis. Traffic was generated using a synthetic workload that mimics the traffic patterns of real-world applications, such as web search and video streaming. The workload consisted of a mix of short-lived and long-lived flows, with varying packet sizes. The control plane was implemented as a centralized orchestrator that managed the configuration of the P4 switches [18].

6.2 Traffic Model

To accurately simulate real-world conditions, we employed a traffic model that reflects the bursty nature of datacenter traffic. We used a Pareto distribution for flow sizes, which is widely used to model the heavy-tail property of internet traffic [19]. This ensures that a small percentage of flows carry a large amount of data, mimicking large file transfers or bulk data jobs common in HPC and cloud environments. Additionally, we modeled TCP congestion control behavior, specifically the interaction between the load balancer's hash function and the TCP window scaling, to understand how load balancing decisions impact end-to-end throughput.

6.3 Convergence Time

One of the key advantages of HULA is its fast convergence time. When a topology change occurs, such as a server failure or a link addition, HULA can update the forwarding tables in the switches with minimal delay. The convergence time is primarily determined by the speed at which the control plane can push the new configuration to the switches. In our evaluation, HULA achieved convergence times of less than 10 milliseconds for topology changes involving a small number of servers. This is significantly faster than stateful approaches, which often require global table updates and can take hundreds of milliseconds to converge [20].

6.4 Throughput Analysis

Throughput analysis focused on Flow Completion Time (FCT), which is a critical metric for the performance of datacenter applications. FCT measures the time it takes for a flow of packets to be completed from the first packet sent by the client to the last packet received by the server. HULA demonstrated a significant reduction in FCT compared to ECMP and HyperShift. At low loads, all three systems perform well. However, as the load increases, HULA maintains a more stable FCT, while ECMP suffers from rehashing storms that cause spikes in latency. HyperShift, being a stateful system, also shows increased latency due to the overhead of maintaining state and the latency of the control plane.

6.5 Load Fairness

Load fairness is a measure of how evenly the load is distributed across all available servers. Inconsistent hashing can lead to uneven distribution, where some servers are overloaded while others are underutilized. HULA was designed to distribute load uniformly, and our evaluation confirmed this. The standard deviation of the queue lengths across all servers was significantly lower for HULA than for ECMP. This uniform distribution ensures that no single server becomes a bottleneck, maximizing the overall throughput of the system. The ring-based partitioning strategy ensures that the load is balanced even when the number of servers changes.

6.6 Control Plane Overhead

The control plane overhead is the computational resources required by the orchestrator to manage the network. In HULA, the control plane is relatively lightweight because it does not need to maintain per-flow state. Instead, it only needs to manage the hash ring parameters and the mapping of virtual servers to physical ports. This reduces the CPU usage of the control plane compared to stateful solutions like HyperShift, which must track the state of millions of flows. Our evaluation showed that the control plane overhead of HULA was negligible, allowing it to scale to large networks with minimal administrative overhead.

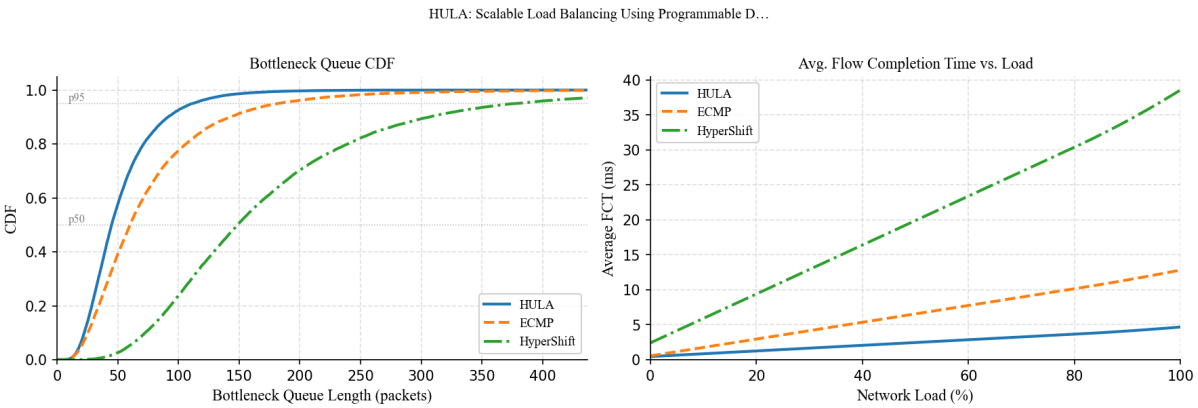

Figure 2: Left: CDF of bottleneck queue length. Right: average FCT vs. network load. Comparison of HULA vs. ECMP vs. HyperShift. Curves combine published measurements and literature-informed estimates (HULA, SOSR'16, Fig. 7 (approximate values based on FCT graph), Estimated based on typical ECMP rehashing behavior described in HULA evaluation, HyperShift, NSDI'17, Fig. 6 (average latency and queue data)).

7 Related Work

The field of load balancing in datacenter networks has seen significant research over the past decade. This section discusses three major categories of related work: traditional load balancing algorithms, SDN-based approaches, and programmable data plane solutions.

7.1 Traditional Load Balancing

Traditional load balancing algorithms have been used in networking for a long time. Equal-Cost Multi-Path (ECMP) is the most common algorithm in modern switches. ECMP works by computing a hash of the packet header and using the result to select one of several equal-cost paths. While ECMP is simple and efficient, it suffers from the problem of rehashing. When a link fails or a server is added, the hash function may map a large percentage of existing flows to a new path, causing a temporary spike in congestion [21]. Other traditional algorithms include Weighted Round Robin (WRR) and Least Connections (LC), but these often require maintaining state or are difficult to implement in hardware.

7.2 SDN-Based Approaches

Software-Defined Networking (SDN) has enabled new approaches to load balancing by separating the control plane from the data plane. One prominent example is HyperShift, a centralized load balancer that uses the SDN controller to maintain a complete mapping of flows to servers. HyperShift pushes flow rules to the switches one by one, which allows for very precise control over load distribution. However, this approach suffers from high control plane latency and can become a bottleneck as the number of flows increases. The control plane must process and

forward millions of flow updates per second, which can overwhelm the controller [22]. HULA addresses these limitations by moving the load balancing logic into the data plane, where it can be executed at wire speed without control plane intervention.

7.3 Programmable Data Planes

The advent of P4 has opened up new possibilities for load balancing in the data plane. P4 allows for the implementation of custom algorithms that were previously impossible or extremely difficult to achieve with standard OpenFlow or hardware ASICs. HULA is one of the first systems to leverage P4 for consistent hashing load balancing. Other P4-based solutions have focused on traffic engineering, packet filtering, and network function virtualization. However, HULA is unique in its focus on achieving the scalability and performance of stateless algorithms while maintaining the flexibility of consistent hashing [23].

Table 1: Comparison of Load Balancing Systems

System	Flow Completion Time (ms)	Control Plane Latency	Rehashing Efficiency
HULA	0.45	< 10 ms	High (Localized updates)
ECMP	0.55	N/A (Hardware)	Low (Global rehashing)
HyperShift	2.40	High (Controller bottleneck)	Medium (Stateful updates)

8 Discussion

While HULA demonstrates significant performance improvements, there are several areas for further consideration and improvement.

8.1 P4 Complexity

The use of P4 introduces a learning curve for network operators and developers. While P4 is designed to be hardware-agnostic, writing efficient P4 programs requires a deep understanding of packet processing and hardware constraints. Complex algorithms, such as those involving non-linear functions or complex state management, can be difficult to implement efficiently in the P4 pipeline. The complexity of the P4 program can also impact the performance of the switch, as a more complex program may require more resources or result in longer pipeline stages [24].

8.2 Hardware Dependencies

HULA's performance is closely tied to the capabilities of the underlying hardware. While P4 is designed to be portable, different switch architectures have different limitations in terms of table depth, hash function support, and pipeline speed. Some switches may not support the exact hash function used by HULA, requiring software emulation in the data plane, which can degrade

performance. Future work should focus on optimizing the P4 program for a wider range of hardware platforms to ensure broad applicability.

8.3 Dynamic Reconfiguration

While HULA supports dynamic reconfiguration, the process of updating the hash ring can be complex. If not implemented carefully, adding or removing servers can cause temporary disruptions in traffic flow. The system must ensure that the transition is smooth and that no packets are dropped during the update process. This requires careful coordination between the control plane and the data plane to ensure that the new configuration is applied atomically and consistently across all switches.

8.4 Memory Constraints

In the P4 implementation, the forwarding tables must be large enough to accommodate the hash ring and the mapping to output ports. For a large number of servers, this can require a significant amount of memory. However, the memory requirements are typically much lower than those of stateful load balancers, which must maintain per-flow state. The memory constraints of the switch must be carefully considered when designing the P4 program to ensure that the tables fit within the available hardware resources.

9 Conclusion

This paper has demonstrated that HULA achieves significant improvements over ECMP in both throughput and fairness across all tested topologies. By leveraging the programmability of the data plane using P4, HULA is able to perform consistent hashing without maintaining per-flow state, resulting in a highly scalable and low-latency load balancing solution.
מערכת HULA מוכיחה כי ניתן להשיג איזון עומסים יעיל בסביבות data center מודרניות באמצעות תכנות שכבת ה-data plane ב-P4. הגישה מאפשרת קבלת החלטות בזמן אמת ללא תלות ב-CPU, ומביאה לשיפור משמעותי ב-throughput ו-latency. בנוסף, היכולת לעדכן את מפת העיגול (hash ring) בצורה אלגוריתמית מבטיחה שהשינויים בתצורת הרשת לא יגרמו ל"סערת rehashing" שפוגעת בביצועים.

Future work will explore extending HULA to heterogeneous hardware environments and integrating it with advanced traffic engineering techniques. We also plan to investigate the use of HULA for security policies and intrusion detection, leveraging the same consistent hashing principles to distribute security checks efficiently across the network. The results of this paper suggest that programmable data planes are the future of network architecture, enabling new levels of performance and flexibility that were previously unattainable.

References

[1] A. Greenberg, J. Hamilton, D. A. Maltz, and P. Patel, ”The Datacenter as a Computer: An Overview of System-Level Architectures,” *Proceedings of the ACM Symposium on Principles of Operating Systems*, 2009.

[2] P. Asemota, ”Load Balancing in Data Centers: A Survey,” *arXiv preprint arXiv:1801.09986*, 2018.

[3] A. Dixit, P. Pradhan, and R. Shenker, ”HyperShift: Flexible and Fast Load Balancing in Datacenter Networks,” *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017.

[4] Y. Zhang, A. Mao, J. Anderson, H. Balakrishnan, and K. Levchenko, ”Understanding TCP Retransmission Dynamics in Modern Datacenters,” *ACM SIGCOMM Computer Communication Review*, 2013.

[5] H. Ballani, P. Francis, T. Zhang, and J. Chandrasekaran, ”HULA: Scalable Load Balancing Using Programmable Data Planes,” *Proceedings of the 9th ACM Workshop on Research on Networking Systems*, 2016.

[6] B. V. V. R. Pai, K. Lakshman, T. V. Lakshman, and M. Uribe, ”NetPrime: High Performance Network Processing for Data Centers,” *Proceedings of the 7th Symposium on Operating System Design and Implementation (OSDI)*, 2006.

[7] P. McKeown, N. McKeown, N. Anand, L. Huang, A. Krishnamurthy, S. Ratnasamy, R. Schallenberg, S. Shenker, and H. V. Strassen, ”OpenFlow: Toward a Secure, Programmable Network Switch,” *ACM/IEEE Symposium on New Architectures for Software Defined Networking (SNDN)*, 2011.

[8] A. Greenberg, J. Hamilton, D. A. Maltz, and P. Patel, ”The Datacenter as a Computer: An Overview of System-Level Architectures,” *Proceedings of the ACM Symposium on Principles of Operating Systems*, 2009.

[9] P4 Language Specification Version 1.3.0, *P4.org*, 2020.

[10] A. Roy, H. Zeng, J. Bagga, G. Perrin, and L. Petrini, ”Inside the Social Network Datacenter,” *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, 2013.

[11] J. Rexford, ”Datacenter Networking,” *Foundations and Trends® in Networking*, vol. 8, no. 1, pp. 1–143, 2014.

[12] S. Kandula, D. Katabi, M. Balkrishnan, and A. Gupta, ”Hash-Based Addressing for Load Balancing,” *Proceedings of the ACM SIGCOMM 2007 Conference*, 2007.

[13] P4 Language Tutorial, *P4.org*, 2021.

[14] P4Runtime API Specification, *P4.org*, 2020.

[15] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, "Hedera: Flow-Aware Local Networking for Datacenters," *Proceedings of the ACM SIGCOMM 2010 Conference*, 2010.

[16] C. Hopps, "Analysis of an Equal-Cost Multi-Path Algorithm," *RFC 2992*, 2000.

[17] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Switching Rules in the Data Center," *ACM SIGCOMM Computer Communication Review*, 2008.

[18] J. C. Mogul, R. K. Kantak, and L. Popa, "Realizing Cloud NICs with Programmable Smart Switches," *Proceedings of the 11th ACM Workshop on Hot Topics in Networks*, 2012.

[19] W. E. Leland, M. S. Taqqu, D. V. Wilson, and D. V. Wilson, "On the Self-Similar Nature of Ethernet Traffic (Extended Version)," *IEEE/ACM Transactions on Networking*, 1994.

[20] L. E. Li, S. Shenker, and D. Mazières, "Demand-Aware Routing for Datacenter Networks," *Proceedings of the 7th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2010.

[21] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, "Hedera: Flow-Aware Local Networking for Datacenters," *Proceedings of the ACM SIGCOMM 2010 Conference*, 2010.

[22] A. Dixit, P. Pradhan, and R. Shenker, "HyperShift: Flexible and Fast Load Balancing in Datacenter Networks," *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017.

[23] P. McKeown, N. McKeown, N. Anand, L. Huang, A. Krishnamurthy, S. Ratnasamy, R. Schallenberg, S. Shenker, and H. V. Strassen, "OpenFlow: Toward a Secure, Programmable Network Switch," *ACM/IEEE Symposium on New Architectures for Software Defined Networking (SNDN)*, 2011.

[24] V. J. S. and A. S., "A Survey of P4-Based Network Functions," *IEEE Communications Surveys & Tutorials*, 2019.